
1. PL/SQL Functions

A **function** in PL/SQL is a stored subprogram that performs an action and **returns a value**.

Syntax of a Function

```
CREATE OR REPLACE FUNCTION function_name (param1 IN data_type)
RETURN return_data_type AS
BEGIN
    -- Your PL/SQL logic here
    RETURN some_value;
END;
/
```

Example: Function to Calculate Bonus

```
CREATE OR REPLACE FUNCTION calculate_bonus(salary IN NUMBER)
RETURN NUMBER AS
    bonus NUMBER;
BEGIN
    bonus := salary * 0.1; -- 10% bonus
    RETURN bonus;
END;
/
```

Calling the Function

```
SELECT calculate_bonus(50000) FROM dual;
```

- ◆ Returns 5000 (10% of 50000).
 - ◆ Can also be used in queries.
-

2. PL/SQL Procedures

A **procedure** is a stored subprogram that performs an action but **does not return a value** (unlike functions).

Example: Procedure to Insert an Employee

```
CREATE OR REPLACE PROCEDURE add_employee (  
    emp_id IN NUMBER,  
    emp_name IN VARCHAR2,  
    emp_salary IN NUMBER  
) AS  
BEGIN  
    INSERT INTO employees (id, name, salary) VALUES (emp_id, emp_name, emp_salary);  
    COMMIT;  
END;  
/
```

Calling the Procedure

```
BEGIN  
    add_employee(101, 'Alice', 50000);  
END;  
/
```

♦ Key Difference Between Function and Procedure:

Aspect	Function	Procedure
Returns a Value?	✓ Yes	✗ No
Can Be Used in SQL Queries?	✓ Yes	✗ No
Used for Business Logic?	✓ Yes	✓ Yes

3. PL/SQL Triggers

A **trigger** is a stored procedure that is **automatically executed** before or after an event (INSERT, UPDATE, DELETE) occurs on a table.

Example: Trigger to Log Employee Salary Changes

```
CREATE OR REPLACE TRIGGER log_salary_changes
AFTER UPDATE ON employees
FOR EACH ROW
BEGIN
    INSERT INTO salary_log (emp_id, old_salary, new_salary, change_date)
    VALUES (:OLD.id, :OLD.salary, :NEW.salary, SYSDATE);
END;
/
```

- ♦ **:OLD** refers to the value before update, and **:NEW** refers to the new value after update.
 - ♦ The trigger **automatically logs salary changes** when the salary is updated.
-

4. DBMS Locks in PL/SQL

Locks are used to **control concurrent access** to data in a database.

Types of Locks

Lock Type	Description
Implicit Lock	Automatically acquired by Oracle when modifying data.
Explicit Lock (DBMS_LOCK)	Manually acquired using <code>DBMS_LOCK.REQUEST</code> .
Row-Level Lock	Acquired when a single row is modified (<code>SELECT ... FOR UPDATE</code>).
Table-Level Lock	Acquired using <code>LOCK TABLE</code> statement.

Example: Explicit Lock Using DBMS_LOCK

```
DECLARE
    lock_handle VARCHAR2(128);
    lock_status NUMBER;
BEGIN
    DBMS_LOCK.ALLOCATE_UNIQUE ('my_lock', lock_handle);
    lock_status := DBMS_LOCK.REQUEST(lock_handle, DBMS_LOCK.X_MODE, 5, TRUE);

    IF lock_status = 0 THEN
        -- Critical Section: Execute code that requires exclusive access
        DBMS_OUTPUT.PUT_LINE('Lock acquired successfully');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Failed to acquire lock');
    END IF;
END;
/
```

- ♦ **DBMS_LOCK.REQUEST** is used for manually handling locks in PL/SQL.
 - ♦ Helps prevent **race conditions** in concurrent transactions.
-

5. ROLLBACK and COMMIT in PL/SQL

These commands control **transactions** in PL/SQL.

COMMIT

- **Saves all changes** made by the transaction.
- **Cannot be undone** after commit.
- Automatically issued when a session ends.

Example: Using COMMIT

```
BEGIN
    INSERT INTO employees (id, name, salary) VALUES (102, 'Bob', 45000);
    COMMIT; -- Saves changes permanently
END;
/
```

ROLLBACK

- **Reverts changes** made in the current transaction.
- Useful for **undoing mistakes**.

Example: Using ROLLBACK

```
BEGIN
  INSERT INTO employees (id, name, salary) VALUES (103, 'Charlie', 40000);
  ROLLBACK; -- Undo changes
END;
/
```

- ♦ The **INSERT** statement is **undone** because of **ROLLBACK**.

6. SAVEPOINT

- Allows rolling back **only part of a transaction**.
- Multiple savepoints can be used.

Example: Using SAVEPOINT

```
BEGIN
  INSERT INTO employees (id, name, salary) VALUES (104, 'David', 55000);
  SAVEPOINT sp1;

  UPDATE employees SET salary = 60000 WHERE id = 104;
  SAVEPOINT sp2;

  DELETE FROM employees WHERE id = 104;

  ROLLBACK TO sp2; -- Undo DELETE but keep the salary update
  COMMIT; -- Save remaining changes
END;
/
```

- ♦ **ROLLBACK TO sp2** undoes the DELETE but keeps the UPDATE.
 - ♦ **COMMIT** ensures all final changes are saved.
-

Summary

PL/SQL Concept	Description
Functions	Return a value and can be used in queries.
Procedures	Perform actions but do not return values.
Triggers	Execute automatically on INSERT, UPDATE, DELETE events.
DBMS Locks	Manage concurrent access to prevent conflicts.
ROLLBACK	Reverts changes since last COMMIT or SAVEPOINT .
COMMIT	Saves all changes permanently.
SAVEPOINT	Marks a rollback point within a transaction.
